AI Critique

The helper AIs were useful because they quickly created broad lists of positive, negative, boundary, security, state, and schema tests. However, they were not fully correct. For FR-03, the AI assumed that email matching should be case-insensitive even though the specification did not say this. It also accepted the implementation's different response for an unknown email before I identified the account-enumeration risk. For FR-09, it created a zero-value coupon case without first checking that coupon creation requires a positive discount. For FR-12, it focused on normal invalid signatures but missed the more dangerous link between the public hard-coded JWT secret and a correctly signed forged admin token.

The AI also missed several cases involving time or more than one request. Examples are concurrent OTP reuse, coupon usage races, stale roles after a token is issued, deleted-user tokens, and request rate limits. These failures happened because the prompt described endpoints and common security rules, while the model usually reasoned about one request at a time. It did not automatically trace every related endpoint or combine information from source code, database state, and security requirements.

I learned that collaborating with AI needs small, controlled steps. First, I should give a narrow feature and clear coverage goals. Second, I must ask for assumptions and keep them visible. Third, I must audit every expected result against the real specification. Finally, I should add human tests based on abuse flows, concurrency, and state after the response. AI is a coverage assistant, not the test oracle. The student remains responsible for deciding what is correct and for checking that generated tests can really detect faults.